Solucionario y guion de la sesión 1

UD1 · Desarrollo del software · Documento del profesor

Índice

1 Reparto de los 120 minutos **1**

2 E1 · Arranque en frío **3**

3 E2 · Clasificar **3**

4 E3 · Por qué no arranca **3**

5 E4 · Programa o aplicación **5**

6 E5 · Ordena la cadena **5**

7 P1.1 · Mirar dentro de un binario **6**

7.1 Firmas que van a encontrar 6

7.2 El momento de la clase 6

7.3 Si alguien termina pronto 6

8 E6 · La relectura **7**

9 Material que hay que llevar preparado **7**

1 Reparto de los 120 minutos

Hora	Bloque	Min.
0:00	Portada y plan de la sesión	5
0:05	**E1 · Arranque en frío**	5
0:10	A · Software y hardware. Las capas del sistema	11
0:21	**E2 · Clasificar**	6
0:27	B · Tu código nunca toca el disco	8
0:35	**E3 · Por qué no arranca**	6
0:41	Idea: la portabilidad la porta alguien	2
0:43	**DESCANSO**	10
0:53	C · Programa, aplicación, a medida vs estándar	6
0:59	**E4 · Programa o aplicación**	5
1:04	D · Fuente, objeto y ejecutable. La cadena en C	15
1:19	**E5 · Ordena la cadena**	6
1:25	**P1.1 · Mirar dentro de un binario**	27
1:52	**E6 · Relectura del E1**	6
1:58	Para el martes	2

Ojo

Si vas mal de tiempo, recorta la P1.1, nunca el E6. La P1.1 se puede dejar en 15 minutos haciendo solo los pasos 2 y 3 en el proyector, todos a la vez. El E6 es el que cierra el arco de la sesión y sin él el E1 no ha servido para nada.

2 E1 · Arranque en frío

No lo corrijas. La tentación de dar la respuesta ahora es fuerte y arruina el cierre. Di solo «guardadlo, esto lo volvemos a leer al final».

Pasea por el aula sin ayudar. Lo que vas a leer por encima de los hombros te da el mapa del grupo:

Lo que escriben	Qué te dice
«Se abre el programa»	No tiene ningún modelo mental. Es lo normal
«El sistema operativo lo carga en memoria»	Ha dado algo de sistemas o ha leído
Menciona bibliotecas, enlazado o procesos	Ha programado en C o viene de otro ciclo

3 E2 · Clasificar

Elemento	Respuesta	Observación
Controlador de impresora	Sistema	Clara
Google Chrome	Aplicación	Aunque lleve DevTools dentro
Visual Studio Code	Programación	Clara
UEFI	Sistema	Muchos no saben qué es: explícalo
Photoshop	Aplicación	Clara
npm	Programación	Gestor de dependencias
El antivirus	**Sistema**	**Discutible**
Node.js	**Programación**	**Discutible**
El compilador de C	Programación	Clara
El Buscaminas	Aplicación	Clara

Idea clave

Los tres dudosos son el ejercicio de verdad. No des la solución como un dogma:

- **El antivirus.** Mucha gente lo pone en aplicación porque tiene ventana e icono. Lo defendible es sistema, porque se engancha al núcleo e intercepta operaciones de fichero. Si alguien argumenta lo contrario con criterio, dalo por bueno.
- **Node.js.** Es un entorno de ejecución. Como herramienta con la que desarrollas, es de programación; como cosa que tiene que estar instalada para que un programa funcione, se parece bastante a software de sistema. Las dos defensas valen.
- **npm.** Nadie lo discute, pero pregunta igualmente qué es exactamente. La mitad lo usa sin saber qué hace.

El criterio de evaluación pide justificar, no acertar la etiqueta.

4 E3 · Por qué no arranca

Con dos razones basta. Las tres buenas:

1. **El formato del ejecutable es distinto.** Windows usa PE (empieza por los bytes `4D 5A`, «MZ») y Linux usa ELF (`7F 45 4C 46`). El sistema ni siquiera reconoce el fichero como algo ejecutable suyo. → *capa de sistema*.
2. **Las llamadas al sistema son distintas.** El `.exe` le pide cosas a la API de Windows, y esas funciones no existen en Linux. → *frontera entre modo usuario y núcleo*.
3. **Faltan las bibliotecas.** Las DLL que necesita no están. → *enlace dinámico*.

Ojo

El error que va a salir seguro: «porque el procesador es distinto». Es falso y merece dos minutos. Los dos sistemas corren sobre x86-64: **el código máquina es el mismo**. Lo que cambia es el envoltorio del fichero y a quién le pide las cosas el programa.
Este matiz es el que de verdad no entienden, y es el que hace que luego comprendan qué es una máquina virtual y qué hace WSL.

El extra. Un `.js` no es un ejecutable: es texto. Lo que se ejecuta es Node, y de Node hay una versión compilada para cada sistema. *Lo portable no es tu código: es el intérprete.* Esta frase es la bisagra hacia el bloque de la portabilidad.

5 E4 · Programa o aplicación

Elemento	Respuesta
El comando `ping`	Programa
WhatsApp en el móvil	Aplicación
Script que renombra 200 fotos	Programa
Moodle	Aplicación
Servicio de cola de impresión	Programa (servicio)
Microsoft Office	Aplicación, y además suite

La última es la buena, y por eso está puesta. La Moodle del instituto es software **estándar** —es un producto genérico que usan miles de centros— que está **configurado y personalizado** para la Junta. Esa es exactamente la situación de casi todo el software que se compra: ni a medida puro ni enlatado puro.

Si sale el debate «pero está tan personalizado que es a medida», déjalo correr un minuto. La respuesta es que **configurar no es desarrollar**.

6 E5 · Ordena la cadena

1. El orden:

`hola.c` → **preprocesador** → `hola.i` → **compilador** → `hola.s` → **ensamblador** → `hola.o` → **enlazador** → `hola.exe`

2. `stdio.h` entra en el preprocesador. Y aquí está el matiz que separa un 7 de un 10: lo que entra es la **declaración** de `printf` (su nombre y sus parámetros), no su código. El código de `printf` llega mucho más tarde, en el **enlazado**, desde la biblioteca estándar de C.

3. Deja de leerse después del ensamblador. El `.s` todavía es texto —ensamblador, pero texto—. El `.o` ya es binario.

4. Ninguno. Es la pregunta trampa. Cuando escribes `node hola.js` no se genera ningún fichero intermedio que tú puedas ver. Node hace su propia traducción por dentro, en memoria, mientras ejecuta: eso es la compilación **JIT**, y se ve en detalle en la sesión 3.

Idea clave

Si alguien pregunta «¿entonces JavaScript es compilado o interpretado?», no lo contestes. Dile que anote la pregunta y que no te deje escaquearte el martes. Es exactamente el contenido 1.5.

7 P1.1 · Mirar dentro de un binario

7.1 Firmas que van a encontrar

Bytes	Como texto	Qué es
`4D 5A`	`MZ`	Ejecutable de Windows (formato PE)
`7F 45 4C 46`	`.ELF`	Ejecutable de Linux
`89 50 4E 47`	`.PNG`	Imagen PNG
`25 50 44 46`	`%PDF`	Documento PDF
`50 4B 03 04`	`PK..`	Fichero ZIP
`CA FE BA BE`	—	Fichero `.class` de Java
`FF D8 FF`	—	Imagen JPEG

Dos cosas que contar mientras trabajan:

- **`MZ` son las iniciales de Mark Zbikowski**, un ingeniero de Microsoft que diseñó el formato en 1983. Cuarenta y tres años después, todos los ejecutables de Windows siguen empezando por sus iniciales.
- **`PK` son las de Phil Katz**, el creador del formato ZIP. Y por eso el paso 5 del ejercicio tiene truco: un `.docx` empieza por PK porque **es un ZIP**. También lo son `.xlsx`, `.odt`, `.jar` y `.epub`. Si alguien lo descubre solo, que lo cuente él.

7.2 El momento de la clase

Déjalo para los **últimos cinco minutos** de la práctica, y prepáralo antes de entrar en el aula:

1. Coge un `.png` cualquiera y haz dos copias: una renombrada a `.exe` y otra a `.txt`.
2. Enséñalas en el proyector. Windows muestra tres iconos distintos y dice que una es un programa.
3. Ábrelas con el editor hexadecimal: las tres empiezan por `89 50 4E 47`. **Las tres son la misma imagen.**

La conclusión la tienen que escribir ellos, y es esta: **la extensión es una etiqueta para las personas, no para la máquina.** El contenido es lo que manda.

De ahí se sale en línea recta hacia por qué llegan virus en ficheros que parecen fotos, y hacia por qué Windows oculta las extensiones por defecto, que es una decisión de diseño bastante discutible. Si te sobran dos minutos, es una conversación que engancha.

7.3 Si alguien termina pronto

- Que busque dentro de `notepad.exe` el texto de algún mensaje de error.
- Que abra un `.docx` con el editor hexadecimal, lo renombre a `.zip` y lo descomprima.
- Que compare el tamaño de `hola.c`, `hola.o` y `hola.exe`. La sorpresa es que el ejecutable pesa muchísimo más que el fuente: ahí está la biblioteca enlazada.

8 E6 · La relectura

Esta es la que no se recorta.

La versión correcta, para comentarla después —no se la des escrita, que la construyan entre todos—:

> Al hacer doble clic, el sistema operativo lee la cabecera del fichero y comprueba que es un ejecutable de su formato. Reserva memoria para él, carga el código, resuelve y carga las bibliotecas dinámicas que necesita, y le dice a la CPU que empiece a ejecutar por la primera instrucción. A partir de ahí, el programa le pide al núcleo, mediante llamadas al sistema, todo lo que necesita: abrir ficheros, dibujar una ventana, usar la red.

Palabras que deberían aparecer en la segunda versión y no en la primera: **formato**, **memoria**, **bibliotecas**, **llamada al sistema**, **núcleo**.

Idea clave

Guarda tú también una copia. En la sesión 4, antes de la prueba escrita, se vuelve a hacer el mismo ejercicio. Ver las tres versiones seguidas es, con diferencia, lo que más motiva de toda la unidad, y a ti te sirve como evidencia de progreso para la evaluación del RA1.

9 Material que hay que llevar preparado

- ☐ La extensión **Hex Editor** probada en el ordenador del aula.
- ☐ Un `.png`, un `.pdf` y un `.zip` accesibles para todos (en Moodle, por ejemplo).
- ☐ Las tres copias del PNG renombradas a `.exe` y `.txt`, hechas de antemano.
- ☐ Comprobado que se puede abrir `notepad.exe` (o `/bin/ls`) desde los equipos del aula.
- ☐ Si vas a hacer la demo de `gcc -E hola.c | wc -l`, comprobado que hay `gcc`.
- ☐ La hora exacta de vuelta del descanso, escrita en la pizarra.